

Các mô hình rời rạc kinh điển (Classic Discrete Models)

Tối ưu hoá – AI203DV01
(Optimization)
Vũ Đình Khôi

Nội dung

- Giới thiệu
- Một số bài toán
 - Minimum set cover
 - Set packing
 - Bin packing
 - Travelling salesman problem (TSP)

- Chapter 05. Serge Kruk. (2018). *Practical Python AI Projects: Mathematical Models of Optimization Problems with Google OR-Tools*. Apress
- K. Lian, "Hands-on Optimization with OR-Tools in Python," 2023. [Online]. Available: <https://kunlei.github.io/python-ortools-notes/>.
 - <https://kunlei.github.io/python-ortools-notes/ip-tsp.html>

**I find that I don't understand things unless I try
to program them.**



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- **Richard Feynman**

- Trái ngược với các mô hình liên tục (continuous models) là các mô hình rời rạc (discrete models)
 - Các ràng buộc tuyến tính, các hàm mục tiêu tuyến tính, cùng với các biến chỉ nhận các giá trị nguyên (integral values)
- Tên gọi khác
 - Quy hoạch nguyên: integer programs (IP)
 - Quy hoạch tuyến tính rời rạc: discrete linear programs
- Để mô tả bài toán rất đơn giản, nhưng không phải lúc nào cũng đơn giản để mô hình và giải. Do đó, cần để ý các yếu tố sau:
 - Nhiều bài toán thực tế bao gồm các bài toán thuần túy và đơn giản → làm sao nhận diện chúng?
 - Nhiều bài toán cần các “mẹo” (trickery) để thiết lập mô hình hiệu quả

- Lưu ý, khác với các mô hình giả-nguyên (pseudo-integral) các chương trước không đảm bảo hoàn toàn tính chất nguyên (integrality).
- Tại sao phải yêu cầu các biến nguyên?
 - Liên quan đến các đối tượng đếm được (counting objects) trong thực tế như người, phương tiện, khác với các đối tượng như nước, phần trăm...
 - Hoặc các biến quyết định chỉ biểu diễn các giá trị nhị phân (boolean conditions) như: true/false, yes/no, 0/1,... Hay còn gọi là các biến chỉ thị (indicator variables)
- Chúng ta bắt đầu xem xét một số bài toán kinh điển thú vị sau
 - Minimum set cover
 - Set packing
 - Bin packing
 - Travelling salesman problem (TSP)

- Một số solver cho bài toán mixed integer programming
 - CBC_MIXED_INTEGER_PROGRAMMING or CBC
 - SCIP_MIXED_INTEGER_PROGRAMMING or SCIP
 - BOP_INTEGER_PROGRAMMING or BOP
 - SAT_INTEGER_PROGRAMMING or SAT or CP_SAT
 - GUROBI_MIXED_INTEGER_PROGRAMMING or GUROBI or GUROBI_MIP
 - CPLEX_MIXED_INTEGER_PROGRAMMING or CPLEX or CPLEX_MIP
 - XPRESS_MIXED_INTEGER_PROGRAMMING or XPRESS or XPRESS_MIP
 - GLPK_MIXED_INTEGER_PROGRAMMING or GLPK or GLPK_MIP
- Tạo các đối tượng (instance) của solver
 - `solver = pywraplp.Solver.CreateSolver('CBC')`

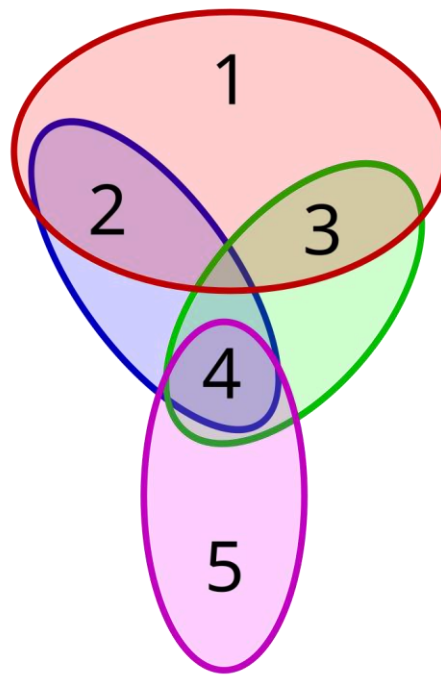
- Có 3 cách khai báo biến nguyên (integer variables) trong OR-tools
 - The `Var(lb, ub, integer: bool, name)` function
 - The `IntVar(lb, ub, name)` function
 - The `Variable.SetInteger(integer: bool)` function

Minimum set cover

■ Bài toán phủ tập hợp tối thiểu

- Cho tập vũ trụ (universe/universal set) $U = \{1, 2, 3, 4, 5\}$
- Và tập hợp các tập con (set of subsets) $S = \{ \{1, 2, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\} \}$
- Tìm số lượng tối thiểu các tập con S sao cho hội (union) của các tập con này bằng tập U (The union of S is equal to U).

→ $U = \{ \{1, 2, 3\}, \{4, 5\} \}$



Minimum set cover

- Một dây chuyền lắp ráp xe hơi điện cần tất cả các thiết bị/linh kiện/phụ tùng (part numbers) để hoàn thành.
- Danh sách các nhà cung cấp (suppliers) với khả năng cung ứng một số thiết bị/phụ tùng
 - Các suppliers khác nhau có thể cung cấp cùng một loại thiết bị/phụ tùng
- Để tiết kiệm chi phí hợp đồng/thầu → Làm sao tối thiểu số lượng các suppliers nhưng vẫn cung cấp đủ thiết bị để hoàn thành việc lắp ráp xe hơi điện.
 - Tìm số lượng ít nhất các suppliers nhưng vẫn đảm bảo đủ (cover) thiết bị/phụ tùng

Minimum set cover

- Bảng mô tả từng suppliers và các thiết bị có thể cung cấp

Supplier	Part Numbers	Supplier	Part Numbers
S0	{ 3; 4; 5; 8; 24 }	S1	{ 11; 15; 21; 23 }
S2	{ 9; 15; 24 }	S3	{ 9; 13 }
S4	{ 5; 11; 12; 14; 16; 20 }	S5	{ 8; 11; 12; 15; 21 }
S6	{ 1; 4; 18; 20 }	S7	{ 0; 3; 6; 11; 13; 15; 21; 23 }
S8	{ 14; 16; 18; 19; 23 }	S9	{ 2; 7; 16; 22 }
S10	{ 10; 14; 21 }	S11	{ 6; 19 }
S12	{ 4; 10; 24 }	S13	{ 3; 4; 7; 9; 17 }
S14	{ 1; 3; 5; 6; 15; 18; 19; 20; 23 }		

Minimum set cover: constructing a model

■ Decision variables

- Với mỗi supplier cần một biến chỉ thị (binary/indicator variables) để biết supplier đó có nằm trong danh sách được ký hợp đồng cung cấp thiết bị hay không?
- S là tập hợp các suppliers
- Mảng các binary variables S_i . Ví dụ, $S_3 = 1$, $S_4 = 0$
 $S_i \in \{0, 1\} \quad \forall i \in S$

■ Objective

- Tối thiểu danh sách các nhà cung cấp

$$\min \sum_{i \in S} S_i$$

- Trong trường hợp mỗi nhà cung cấp có chi phí (cost) thì

$$\min \sum_{i \in S} C_i S_i$$

Minimum set cover: constructing a model

■ Constraints

- Ở khía cạnh tổng quát, chỉ có một ràng buộc duy nhất là: công ty phải truy xuất đến tất cả các loại phụ tùng theo yêu cầu cho một chiếc xe hơi điện.
- Làm sao để đảm bảo có tất cả các loại phụ tùng (tập hợp đầy đủ phụ tùng với số nhà cung cấp tối thiểu)?
- Ví dụ, phụ tùng 23 (part 23) có 4 nhà cung cấp 1, 7, 8, 14. Vậy chọn nhà cung cấp nào?
→ $S_1 + S_7 + S_8 + S_{14}$ tối thiểu là 1
- Tại sao không là 1? → vì có thể có từ 2 nhà cung cấp trở lên cùng cung cấp part 23
- Ví dụ: S_1 và S_7 cùng cung cấp part 15 và part 23 → có thể chọn S_1 cho part 15, S_7 cho part 23 chẳng hạn.

Minimum set cover: constructing a model

■ Constraints

- Với mỗi part $j \rightarrow$ lọc ra tất cả các supplier có cung cấp part j và chọn ít nhất 1 supplier
- Vậy có tất cả n ràng buộc (với n là tổng số lượng các part number)

$$\sum_{i, j \in P_i} S_i \geq 1 \quad j \in P$$

Minimum set cover: executable model

```
1  # Bài toán Minimum Set Cover (set_cover.py)
2
3  from ortools.linear_solver import pywraplp
4  from my_or_tools import *
5
6
7  # D: mảng 2 chiều gồm các part number của mỗi supplier
8  # C: mảng chi phí (cost) của các suppliers
9  def solve_set_cover(D, C=None):
10     solver = pywraplp.Solver('Minimum Set Cover',
11                               pywraplp.Solver.CBC_MIXED_INTEGER_PROGRAMMING)
12     n_suppliers = len(D)
13     # Vì part number theo số thứ tự từ 0, 1, 2, ... n
14     # số lượng part number chính là số lớn nhất trong mảng D cộng thêm 1 (do có part 0)
15     n_parts = max(max(row) for row in D) + 1
```

Minimum set cover: executable model

```
17     # Decision variables
18     S = [solver.IntVar(0, 1, 'S[%d]' % i) for i in range(n_suppliers)]
19
20     # Constraints
21     # Lọc ra tất cả các supplier có cung cấp part j và chọn ít nhất 1 supplier
22     for j in range(n_parts):
23         solver.Add(sum(S[i] for i in range(n_suppliers) if j in D[i]) >= 1)
24
25     # Objective function
26     if C is None:
27         solver.Minimize(solver.Sum(S))
28     else:
29         solver.Minimize(solver.Sum([S[i] * C[i] for i in range(n_suppliers)]))
30
31     status = solver.Solve()
32     Suppliers = [i for i in range(n_suppliers) if S[i].solution_value() > 0]
33     Parts = [[i for i in range(n_suppliers)
34               if j in D[i] and S[i].solution_value() > 0] for j in range(n_parts)]
35
36     return status, ObjVal(solver), Suppliers, Parts
```


Minimum set cover: executable model

```
39 def main():
40     D = [[3, 4, 5, 8, 24],
41          [11, 15, 21, 23],
42          [9, 15, 24],
43          [9, 13],
44          [5, 11, 12, 14, 16, 20],
45          [8, 11, 12, 15, 21],
46          [1, 4, 18, 20],
47          [0, 3, 6, 11, 13, 15, 21, 23],
48          [14, 16, 18, 19, 23],
49          [2, 7, 16, 22],
50          [10, 14, 21],
51          [6, 19],
52          [4, 10, 24],
53          [3, 4, 7, 9, 17],
54          [1, 3, 5, 6, 15, 18, 19, 20, 23]]
```

```
status: 0
obj_val: 7.0
Suppliers: [5, 7, 9, 10, 12, 13, 14]
```

Minimum set cover: executable model

Table 5-2. *Optimal Solution to the Set Cover Problem*

Parts	Suppliers	Parts	Suppliers
All	{ 5; 7; 9; 10; 12; 13; 14 }	Part #0	{ 7 }
Part #1	{ 14 }	Part #2	{ 9 }
Part #3	{ 7; 13; 14 }	Part #4	{ 12; 13 }
Part #5	{ 14 }	Part #6	{ 7; 14 }
Part #7	{ 9; 13 }	Part #8	{ 5 }
Part #9	{ 13 }	Part #10	{ 10; 12 }
Part #11	{ 5; 7 }	Part #12	{ 5 }
Part #13	{ 7 }	Part #14	{ 10 }
Part #15	{ 5; 7; 14 }	Part #16	{ 9 }
Part #17	{ 13 }	Part #18	{ 14 }
Part #19	{ 14 }	Part #20	{ 14 }
Part #21	{ 5; 7; 10 }	Part #22	{ 9 }
Part #23	{ 7; 14 }	Part #24	{ 12 }

Minimum set cover: variations

■ Computer virus detection

- Cơ sở dữ liệu hàng ngàn virus máy tính
- Xây dựng chương trình phát hiện virus (detector)
- Tìm cách xác định các chuỗi bytes ngắn để nhận diện virus hay không
- → tối thiểu các chuỗi bytes đặc trưng để phát hiện tất cả virus máy tính
- → detector chỉ cần dò tìm tập các chuỗi bytes đặc trưng này trong các tập tin dữ liệu

■ Telecommunications

- Xây dựng các trạm phát sóng tại các địa điểm khác nhau sao cho bao phủ hết các khu vực thành phố.
- Biết rằng chi phí xây dựng các trạm phát sóng tại các vị trí là khác nhau (cost)

■ Bài toán Minimum set cover

- Phủ (cover) tập vũ trụ (universal set) với tập các tập con tối thiểu (minimal set of subsets)
→ sẽ có một số phần tử (elements) có thể xuất hiện nhiều hơn một lần trong các tập con.

■ Bài toán Set packing

- Chọn càng nhiều tập con càng tốt nhưng một phần tử không được chọn quá một lần (không xuất hiện trong cùng 2 tập con)
→ sẽ có một số phần tử không được chọn.
- Bài toán lập lịch cho phi hành đoàn (airline crew scheduling)

Set packing: Airline crew scheduling

- Mỗi máy bay phải có một phi công (pilot), một phi công phó (co-pilot), một hoa tiêu (navigator), và một tiếp viên trưởng (purser).
 - Mỗi tập hợp (4 người trên) là một danh sách phi hành đoàn (roster).
- Mỗi pilot
 - Có thể lái một số loại máy bay
 - Có thể đi cùng với các phụ tá họ thích
- Do đó, có thể xem việc kết hợp cụ thể (specific combination) máy bay/phi công/phi công phó/hoa tiêu/tiếp viên trưởng như là
 - Tập con của tập vũ trụ các máy bay và các thành viên phi hành đoàn.
- Bài toán chọn tối đa số lượng các tập con, nhưng không được chọn 2 tập con có cùng một phần tử
 - Vì một pilot không thể có mặt trong 2 phi hành đoàn cùng lúc

Set packing: Airline crew scheduling

Table 5-3. *Example of Set Packing from Crew Scheduling*

Roster #	Crew IDs	Roster #	Crew IDs
0	{ 1 ; 18; 30 }	1	{ 4 24 36 }
2	{ 1; 5; 9 }	3	{ 7; 17; 30 }
4	{ 10; 23; 25 }	5	{ 8; 10; 25 }
6	{ 19; 29; 36 }	7	{ 3; 4; 17 }
8	{ 19; 28; 40 }	9	{ 11; 24; 31 }
10	{ 18; 30; 33 }	11	{ 22; 25; 26 }
12	{ 13; 15; 26 }	13	{ 21; 27; 28 }
14	{ 7; 12; 33 }		

Set packing: Constructing a model

■ Decision variables

- Biến chỉ thị (indicator) cho biết danh sách phi hành đoàn (roster) i được chọn?
- S : tập các crew rosters

$$s_i \in \{0, 1\} \quad \forall i \in S$$

■ Objective

- Tối đa số lượng các rosters được chọn

$$\max \sum_{i \in S} s_i$$

■ Constraints

- Không bao giờ chọn 2 rosters có cùng các thành viên phi hành đoàn

$$\sum_{i: j \in S_i} s_i \leq 1 \quad \forall j \in U$$

19/12/2024

Set packing: Executable model

```
25     # Constraints
26     # Loại ra tất cả các Roster Number có chứa Crew Member ID j
27     # và chọn không quá 1 Roster Number
28     for j in range(n_crews):
29         solver.Add(sum(S[i] for i in range(n_rosters) if j in D[i]) <= 1)
30
31     # Objective function
32     if C is None:
33         solver.Maximize(solver.Sum(S))
34     else:
35         solver.Maximize(solver.Sum([S[i] * C[i] for i in range(n_rosters)]))
36
37     status = solver.Solve()
38     Rosters = [i for i in range(n_rosters) if S[i].solution_value() > 0]
39
40     return status, ObjVal(solver), Rosters
```



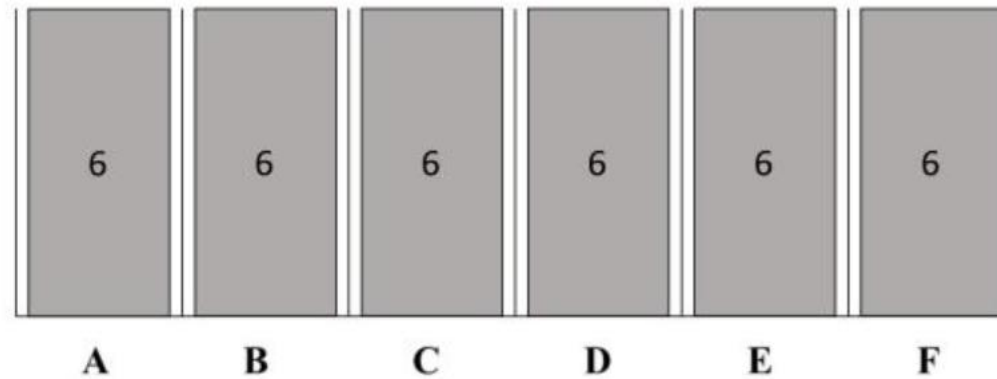
Set packing: Executable model

```
43  def main():
44      # D: mảng 2 chiều gồm các Roster Number và tập các Crew Member ID
45      # (dòng i chính là Roster Number i)
46  D = [[1, 18, 30],
47        [4, 24, 36],
48        [1, 5, 9],
49        [7, 17, 30],
50        [10, 23, 25],
51        [8, 10, 25],
52        [19, 29, 36],
53        [3, 4, 17],
54        [19, 28, 40],
55        [11, 24, 31],
56        [1, 30, 33],
57        [22, 25, 26],
58        [13, 15, 26],
59        [21, 27, 28],
60        [7, 12, 33]]
```

```
status: 0
obj_val: 8.0
Rosters: [2, 4, 6, 7, 9, 12, 13, 14]
```

Bin packing problem

- Bài toán xếp các đồ vật (items) vào hộp/thùng (bin)
 - Cho các vật có khối lượng khác nhau
 - Yêu cầu; xác định số lượng thùng/hộp (với khối lượng giới hạn) tối thiểu



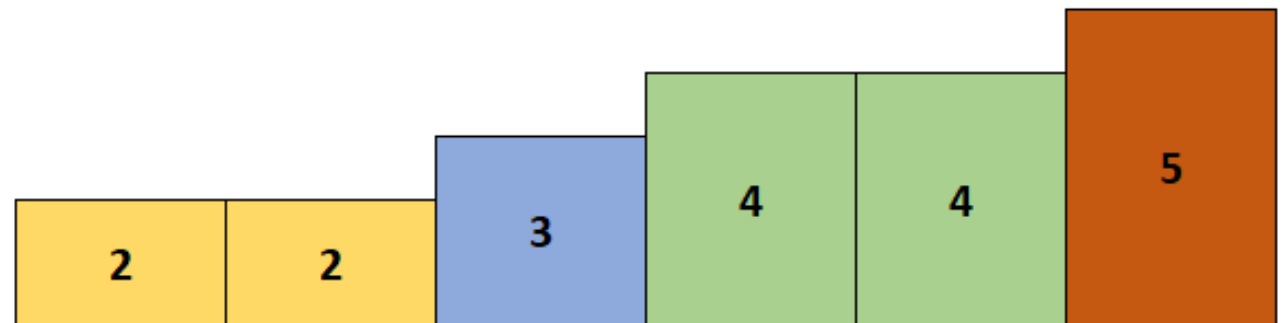
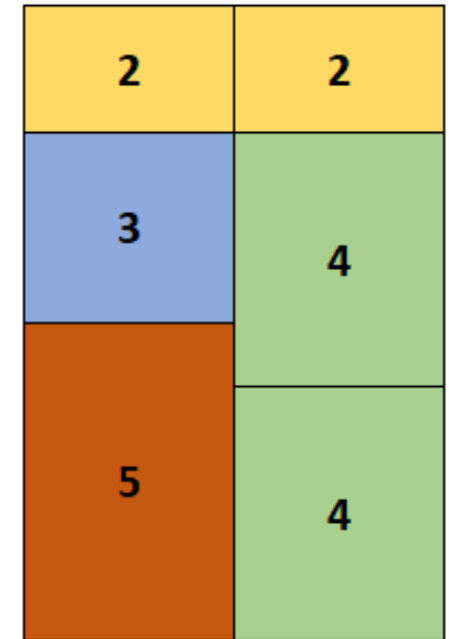
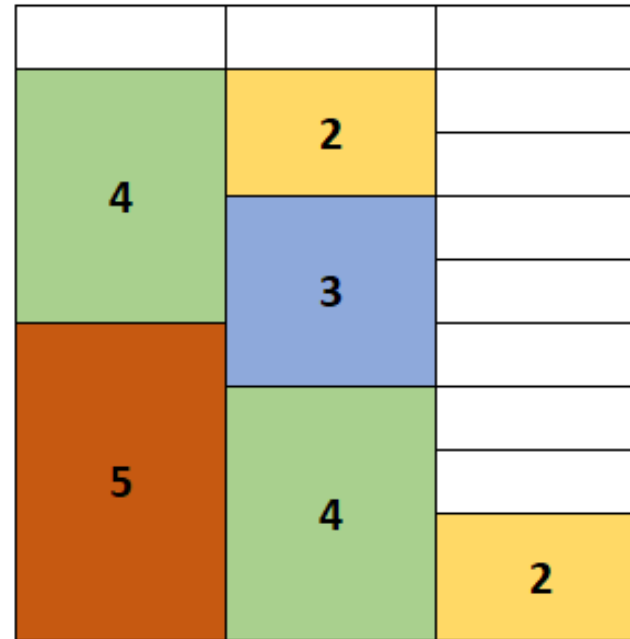
Bin packing problem

■ Hình trái

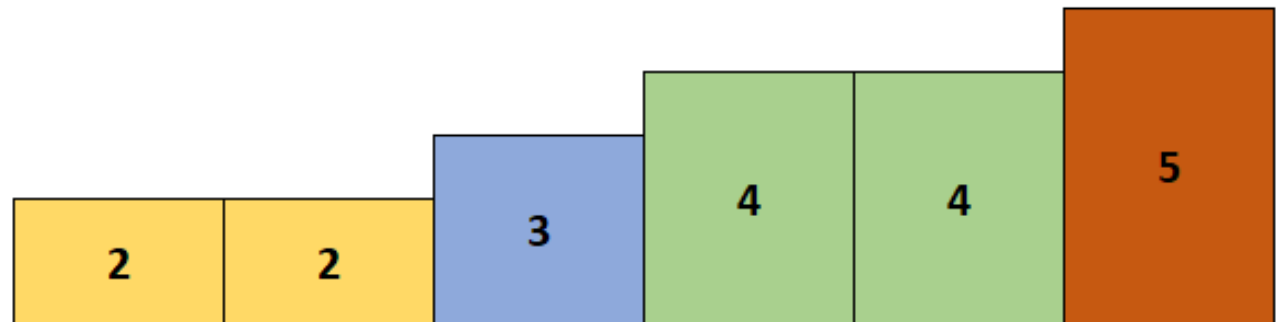
- giải theo thuật toán tham lam (greedy)

■ Hình phải

- Một trong các nghiệm (solution) tối ưu



- Lưu ý các trường hợp hoán vị thứ tự liệt kê các tập con và thứ tự các items trong cùng tập con?



- Bin packing là bài toán phân hoạch một tập hợp (partitioning a set), trong đó:
 - Mỗi phần tử có một trọng số (weight)
 - Sắp xếp các phần tử vào các nhóm nhưng vẫn đảm bảo mỗi nhóm không vượt quá giới hạn cho trước (capacity/prescribed weight limit)
 - Tối ưu hóa (tối thiểu) số lượng các nhóm?
- Bài toán bin packing có nhiều ứng dụng trong thực tế như:
 - Tối ưu sử dụng không gian lưu trữ trong nhà kho (warehouses)
 - Tối thiểu số lượng xe tải vận chuyển hàng
 - Tối ưu sử dụng bộ nhớ máy tính

Bin packing

- Khối lượng vận chuyển tối đa mỗi xe tải (truck weight limit/capacity): 1264
- Các dòng gồm mặt hàng (items), số kiện hàng (number of packages), và khối lượng mỗi kiện hàng.
- Tối ưu (tối thiểu) số lượng xe tải vận chuyển.

Table 5-5. Example of Bin Packing

	Truck Weight Limit	1264
	Number of Packages	Unit weight
0	8	258
1	10	478
2	8	399
Total	26	10036

Item No. 1:
gồm 10 packages
khối lượng mỗi package 478 (kg)

Bin packing: Constructing a model

■ Decision variables

- Kết quả cần biết là “Cần bao nhiêu xe tải?” Và “Kiện hàng nào đi theo xe tải nào?”
- P : tập hợp các packages
- T : tập hợp các trucks

$$x_{i,j} \in \{0, 1\} \quad \forall i \in P, j \in T$$

- $x_{i,j} = 1 \rightarrow$ package i đi cùng/thuộc truck j
- Sử dụng biến indicator để biết xe tải j có được sử dụng hay không?

$$y_j \in \{0, 1\} \quad \forall j \in T$$

■ Objective

$$\min \sum_{j \in T} y_j$$

Bin packing: Constructing a model

■ Constraints

- Mỗi quan hệ giữa các biến $x_{i,j}$ và y_j ?
- Nếu $x_{i,j} = 1$ đối với bất kỳ package i nào đó thì $y_j = 1$ (nghĩa là truck j được dùng để vận chuyển cho ít nhất package i và *package nào đó nếu còn trống chỗ*)

$$x_{i,j} \leq y_j \quad \forall i \in P, \forall j \in T$$

- Tổng khối lượng các package trên truck j không được vượt quá capacity

$$\sum_{i \in P} w_i \times x_{i,j} \leq W_j \quad \forall j \in T$$

- Kết hợp 2 ràng buộc trên:

$$\sum_{i \in P} w_i \times x_{i,j} \leq W_j \times y_j \quad \forall j \in T$$

- Mỗi package thuộc về một truck nào đó

$$\sum_{j \in T} x_{i,j} = 1 \quad \forall i \in P$$

Bin packing: Constructing a model

■ Ý nghĩa của ràng buộc?

$$x_{i,j} \leq y_j \quad \forall i \in P, \forall j \in T$$

- Để ý các biến $x_{i,j}$ và y_j là các indicator (chỉ nhận giá trị 0 hoặc 1).
- Nếu $x_{i,j} = 1 \rightarrow$ tất nhiên truck j được dùng và $y_j = 1$. Do đó: $1 \leq 1$
- Nếu $x_{i,j} = 0 \rightarrow$ package i không đi cùng truck j nhưng có thể có package khác đi cùng truck j . Do đó: $0 \leq 0$ hoặc 1 (hiển nhiên)

Bin packing: Executable model

```
6 def solve_bin_packing(D, W):
7     solver = pywraplp.Solver('Bin Packing Problem',
8         |         |         |         |         |         |         |         |
9         |         |         |         |         |         |         |         |
10        pywraplp.Solver.CBC_MIXED_INTEGER_PROGRAMMING)
11
12     # n_items = len(D)
13     n_packages = sum([item[0] for item in D]) # sum(D[i][0] for i in range(n_items))
14     n_trucks = n_packages # Số lượng xe tải tối đa
15     # Ví dụ chuyển 8 packages có khối lượng 258 kg/package thành dãy 8 phần tử 258
16     weights = []
17     for item in D:
18         weights += item[0] * [item[1]]
19
20     # Decision variables
21     # x[i][j] = 1 nếu package i được đặt vào xe tải j
22     x = [[solver.IntVar(0, 1, f'x[{i}][{j}]')]
23         |         for j in range(n_trucks)] for i in range(n_packages)]
24
25     # y[k] = 1 nếu xe tải k được sử dụng
26     y = [solver.IntVar(0, 1, '') for _ in range(n_trucks)]
```

Bin packing: Executable model

```
25     # Constraints
26     # Mỗi package chỉ được đặt vào 1 xe tải
27     for i in range(n_packages):
28         # hoặc solver.Add(sum(x[i]) == 1)
29         solver.Add(sum(x[i][j] for j in range(n_trucks)) == 1)
30
31     # Khối lượng của các package trên cùng 1 xe tải không vượt quá trọng lượng tối đa
32     for j in range(n_trucks):
33         solver.Add(sum(weights[i] * x[i][j] for i in range(n_packages)) <= W * y[j])
34
35     # Objective
36     min_n_trucks = sum([y[j] for j in range(n_trucks)])
37     solver.Minimize(min_n_trucks)
38
39     # Solve
40     status = solver.Solve()
41
42     return status, ObjVal(solver), SolVal(y)
```

Bin packing: Executable model

```
45  def main():
46      # D: số lượng và khối lượng của các kiện hàng (packages)
47      D = [[8, 258],
48            [10, 478],
49            [1, 399]]
50      # W: maximum capacity (load) of a truck
51      W = 1264
52
53      status, obj_val, y = solve_bin_packing(D, W)
54      print('Status =', status)
55      print('Objective value =', obj_val)
56      print('Solution value =', y)
```

Cần 6 trucks
(tương ứng với 6 giá
trị $y[i] = 1$ bên dưới)

```
Status = 0
Objective value = 6.0
Solution value = [0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0]
```

Bin packing: Executable model

■ Nhận xét

- Một số truck có thể bị bỏ qua (omitted) → mảng $y[]$ có các giá trị 1 không liên tục → có thể 'improve' với ràng buộc $y[i] \geq y[i+1]$
- Các solvers có thể cho nhiều solution khác nhau nhưng objective value (tổng số trucks) là như nhau.

■ Để ý không gian nghiệm khả dĩ (search space of feasible solutions)

- Có thể hoán đổi (swap) toàn bộ số kiện hàng (contents) bên trong 2 truck có cùng max capacity/load
- Có thể hoán đổi 2 packages cùng loại giữa 2 trucks khác nhau
- Hoặc hoán đổi 2 packages cùng loại (nhưng khác thứ tự đưa lên truck) trên cùng truck.

→ Symmetry (tính đối xứng)?

Bin packing: symmetry breaking constraints

- Chạy thử model với input D sau đây
 - $D = [[8, 258], [10, 478], [8, 399]]$
 - Có thể tốn vài giờ tìm nghiệm cho trường hợp D này? Tại sao?
- Symmetry breaking constraints
 - Ngăn ngừa các nghiệm đối xứng → giảm không gian tìm kiếm nghiệm → giảm thời gian xử lý
 - Thêm các ràng buộc (xem code)

Bin packing: symmetry breaking constraints

```
53 # Symmetry breaking
54 if symmetry_breaking:
55     # các truck được chọn lần lượt theo thứ tự (in order)
56     for k in range(n_trucks - 1):
57         solver.Add(y[k] >= y[k + 1])
58
59     # xét từng nhóm các package có cùng khối lượng
60     # nếu một package j được đặt vào một xe tải k, thì các package cùng nhóm với j
61     # nhưng đứng sau j phải được đặt vào xe tải k hoặc trên một trong các xe tải sau k
62     index = 0
63     for i in range(n_groups):
64         lj = index # lower bound
65         uj = index + D[i][0] # upper bound
66         for j in range(lj, uj):
67             for k in range(n_trucks):
68                 # các package cùng nhóm với j nhưng đứng trước j phải được đặt vào xe tải k hoặc trước k
69                 for jj in range(max(lj, j - 1), j):
70                     solver.Add(sum(x[jj][kk] for kk in range(0, k + 1)) >= x[j][k])
71                 # các package cùng nhóm với j nhưng đứng sau j phải được đặt vào xe tải k hoặc sau k
72                 for jj in range(j + 1, min(j + 2, uj)):
73                     solver.Add(sum(x[jj][kk] for kk in range(k, n_trucks)) >= x[j][k])
74         index += D[i][0]
```


Bin packing: symmetry breaking constraints

```
88  ✓ def main():
89      # D: số lượng và khối lượng của các kiện hàng (packages)
90  ✓      D = [[8, 258],
91              [10, 478],
92              [8, 399]]
93      # W: maximum capacity (load) of a truck
94      W = 1264
95
96      print("Solving Bin Packing Problem")
97      print("CBC_MIXED_INTEGER_PROGRAMMING")
98      print("Processing...")
99
100     status, obj_val, x, y = solve_bin_packing(D, W, symmetry_breaking=True)
101     print("Status =", status)
102     print("Objective value =", obj_val)
103     print("Solution value =", y)
```

Bin packing: symmetry breaking constraints

```
104
105     # Print solution
106     weights = get_weights(D)
107     print('Weights:', weights)
108
109     # Các packages trên mỗi truck
110     n = int(obj_val)
111     for j in range(n):
112         print(f"Truck {j}:", [weights[i]
113                               for i in range(len(x)) if y[j] == 1 and x[i][j] == 1])
114
```



Bin packing: symmetry breaking constraints

- Lưu ý nếu không có symmetry breaking
 - → không gian các solutions có thể là $9!$ (nếu hoán vị các giữa các truck)
 - Hoặc trong mỗi truck (vd truck 5) sẽ có $4!$ trường hợp thứ tự các packages (khi load lên truck)

```
Solving Bin Packing Problem
CBC_MIXED_INTEGER_PROGRAMMING
Processing...
Status = 0
Objective value = 9.0
Solution value = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0]
Weights: [258, 258, 258, 258, 258, 258, 258, 258, 478,
Truck 0: [478, 478]
Truck 1: [258, 478, 399]
Truck 2: [399, 399, 399]
Truck 3: [478, 478]
Truck 4: [258, 258, 258, 478]
Truck 5: [258, 258, 258, 478]
Truck 6: [258, 478, 478]
Truck 7: [478, 399]
Truck 8: [399, 399, 399]
```

Bin packing: variations

- Một vài biến thể đơn giản (simpler variations)
- Mỗi xe tải có một tải trọng tối đa khác nhau (different weight capacity)
 - Khi này symmetry-breaking chỉ có thể xem xét trong tập con các xe tải cùng tải trọng tối đa.
- Cố định số lượng xe tải $\rightarrow$ tối đa khối lượng packages vận chuyển
 - Mỗi package ngoài trọng lượng (weight), còn có thêm một giá trị (value) $v_i \rightarrow$ tối đa tổng giá trị (total value)

$$\max \sum_{i \in P} \sum_{j \in T} v_i \times x_{i,j}$$

- Một trong các phiên bản đơn giản hơn của bài toán *bin packing* là *knapsack*
 - Một truck với weight capacity, các packages với value v_i và weight $w_i \rightarrow$ maximize total value?

Bin packing: variations

- Một bài toán có liên hệ gần (closely related problem) là bài toán lập ngân sách (*capital budgeting*)
 - Khoảng thời gian lập kế hoạch nhiều giai đoạn (Multi-period planning horizon): T
 - Tập hợp các dự án khả thi (A set of possible projects): P
 - Mỗi dự án j cần khoảng đầu tư a_{tj} trong giai đoạn t và đại diện một giá trị c_j .
 - Ngân sách giới hạn (given a limited budget) b_t trong giai đoạn t
 - Các dự án nào nên được chỉ định để đầu tư?
- Model là dạng đơn giản của *bin packing*

$$\max \sum_{j \in P} c_j \times y_j$$

$$\sum_{j \in P} a_{tj} \times y_j \leq b_t \quad \forall t \in T$$

- Biến quyết định dự nào nào được chọn để đầu tư: $y_i \in \{0, 1\}$

Bin packing: variations

- Capital budgeting problem
 - Example

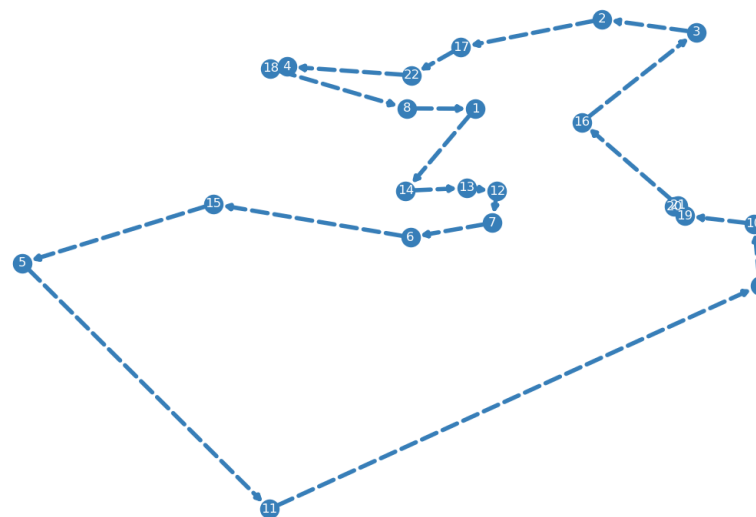
Project (P)	Investment Year 1 (a_{1j})	Investment Year 2 (a_{2j})	Investment Year 3 (a_{3j})	Value (c_j)
A	\$10,000	\$5,000	\$0	\$20,000
B	\$8,000	\$8,000	\$4,000	\$25,000
C	\$7,000	\$7,000	\$6,000	\$22,000
Limited budget (b_t)	\$15,000	\$10,000	\$8,000	

TSP (Travelling Salesman Problem)



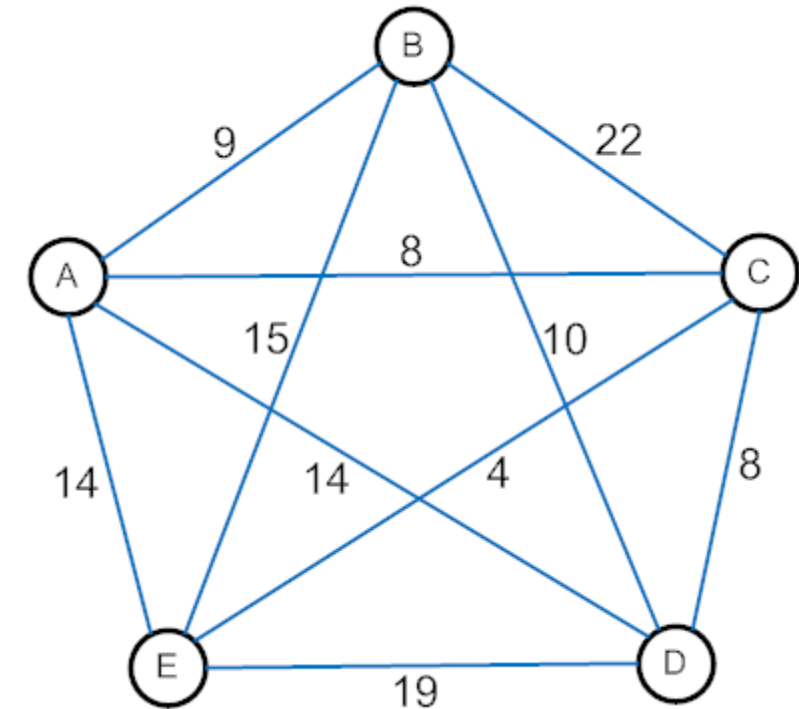
TSP (Travelling Salesman Problem)

- Travelling Salesman Problem (TSP): bài toán người bán hàng
 - Cho trước một danh sách các thành phố và khoảng cách giữa chúng, tìm chu trình ngắn nhất thăm mỗi thành phố đúng một lần.
 - Hay có một người giao hàng cần đi giao hàng tại n thành phố, xuất phát từ một thành phố nào đó, đi qua các thành phố khác để giao hàng và trở về thành phố ban đầu. Tìm quãng đường ngắn nhất để giao hàng?
 - Bài toán NP-khó (Non-deterministic Polynomial Time hard)



TSP (Travelling Salesman Problem)

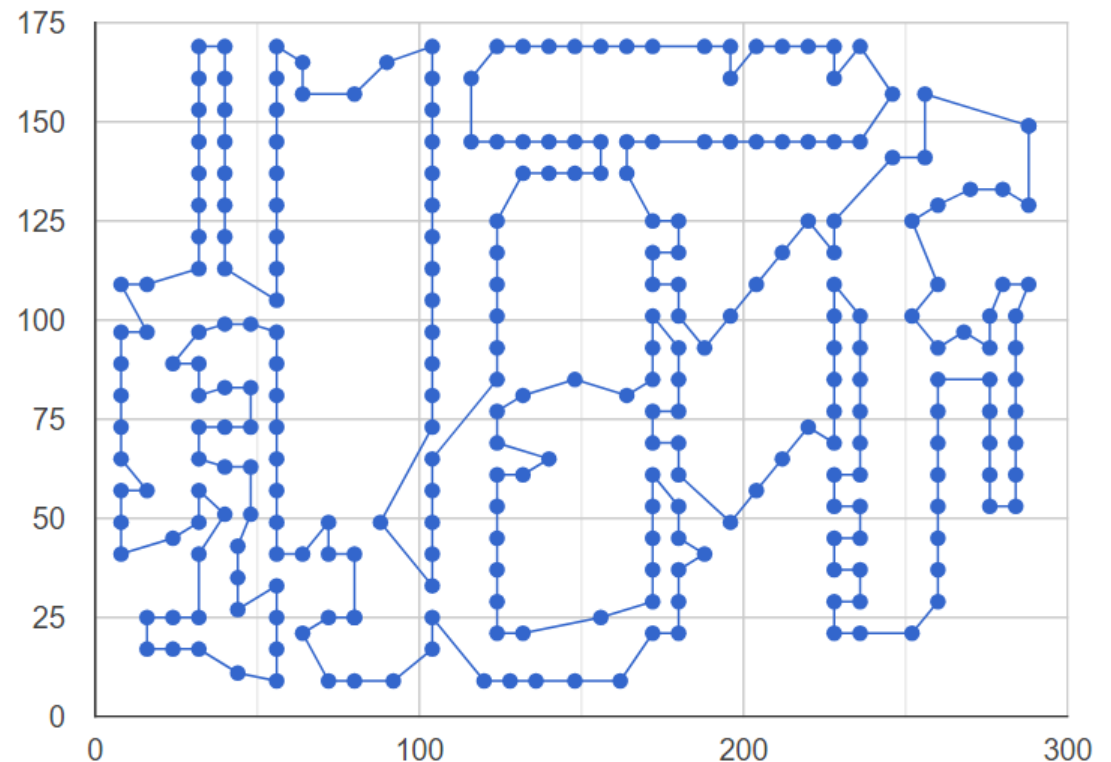
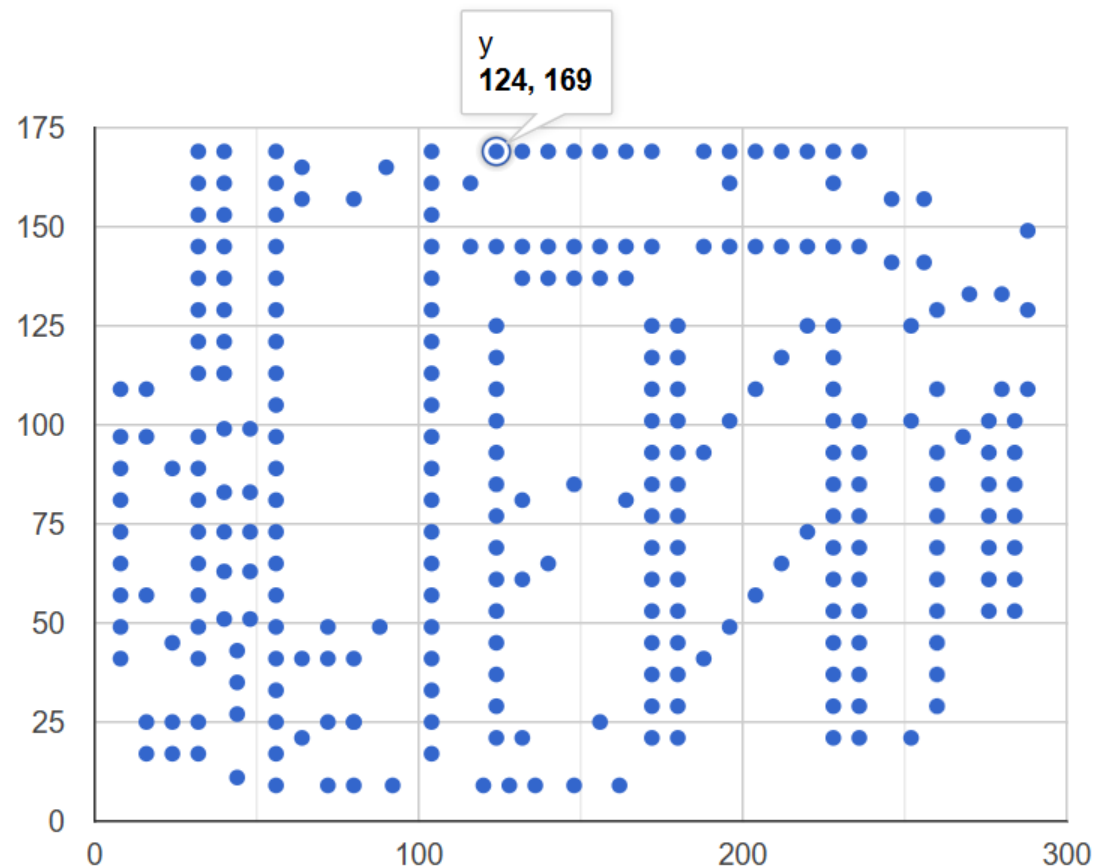
- Dạng đồ thị
- Ứng dụng
 - vehicle routing,
 - logistics and transportation planning,
 - electronic circuit design/circuit board drilling,
 - job sequencing
 - DNA sequencing
 - ...



TSP (Travelling Salesman Problem)

■ Drilling a circuit board

- Mỗi lỗ có vị trí (x, y) → Máy khoan tự động các lỗ (holes) trên bo mạch → tìm đường đi của mũi khoan ngắn nhất (mũi khoan sẽ di chuyển ít nhất)



TSP (Travelling Salesman Problem)

■ Cho ma trận khoảng cách giữa các đỉnh (D)

- Tìm chu trình qua tất cả các đỉnh với tổng khoảng cách nhỏ nhất (find a tour of all vertices minimizing total distance)

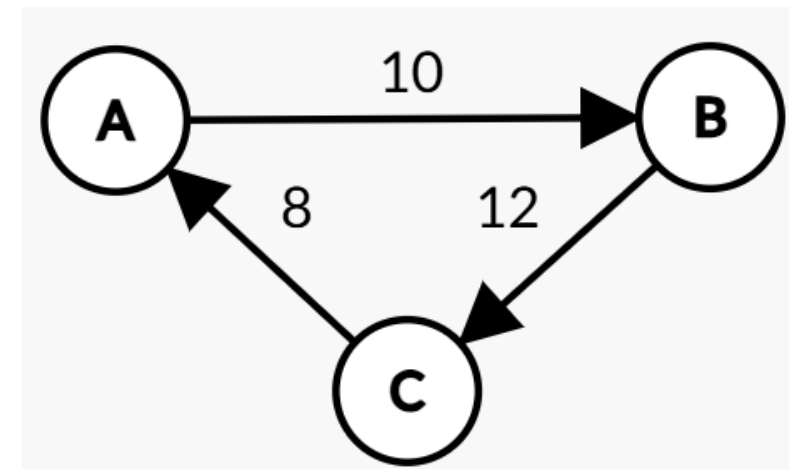
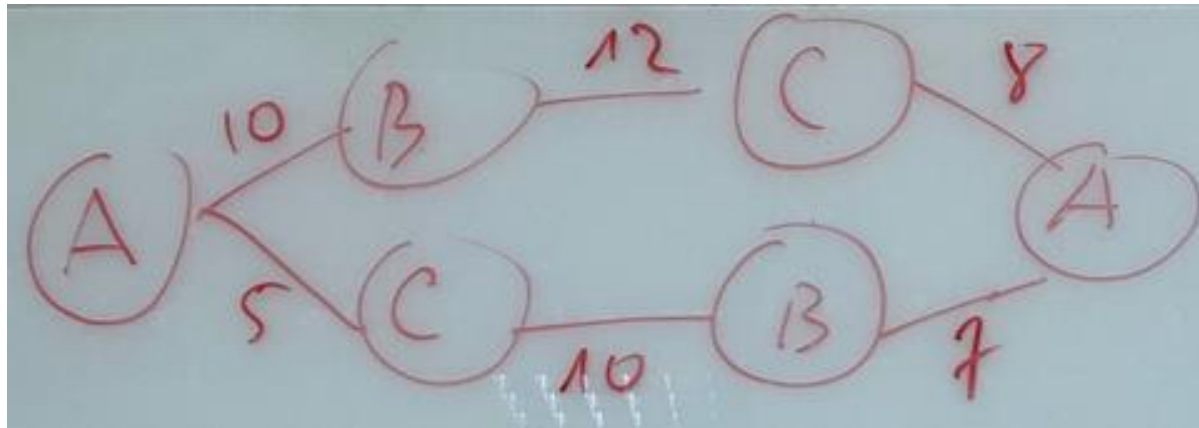
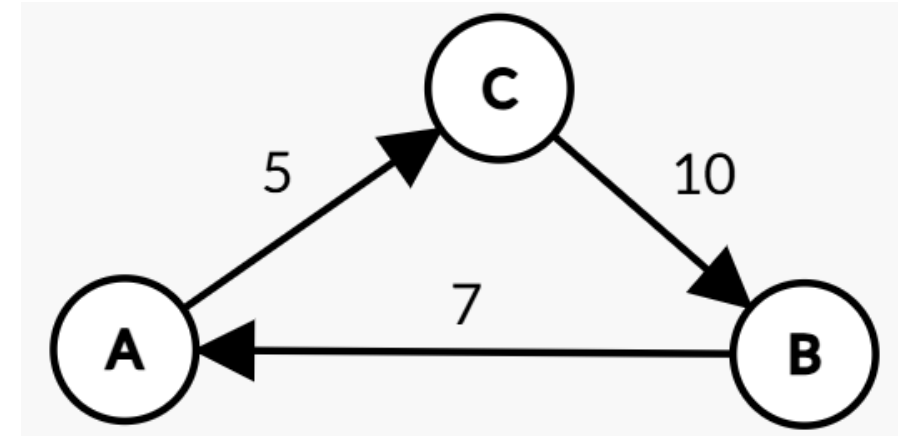
Table 5-9. Example of Distance Matrix for TSP

P (x y)	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9
P0 72 19		711	107	516	387	408	539	309	566	771
P1 10 37	539		769	881	380	546	655	443	295	1140
P2 77 31	122	752		281	441	264	318	448	588	730
P3 89 61	519	875	274		435	334	93	776	949	302
P4 51 61	484	561	338	419		118	268	607	495	431
P5 57 52	409	406	244	380	93		295	544	549	494
P6 82 69	479	735	334	101	345	247		679	809	238
P7 52 1	221	444	433	744	487	435	649		325	840
P8 21 14	510	303	599	984	531	553	847	350		1001
P9 88 96	663	989	664	335	588	434	297	1093	1012	

TSP (Travelling Salesman Problem)

Points	A	B	C
A		10	5
B	7		12
C	8	10	

Decision Variables	A	B	C
A	0	0	1
B	1	0	0
C	0	1	0



TSP: constructing a model

■ Decision variables

- Tương tự như bài toán tìm đường đi ngắn nhất (shortest path problem)
- Cho P là tập hợp các điểm, $x_{i,j}$ là đường nối giữa điểm i và j
$$x_{i,j} \in \{0, 1\} \quad \forall i \in P, \forall j \in P$$

■ Objective

- Tương tự mô hình tìm đường đi ngắn nhất

$$\min \sum_i \sum_j D_{i,j} \times x_{i,j}$$

TSP: constructing a model

■ Constraints

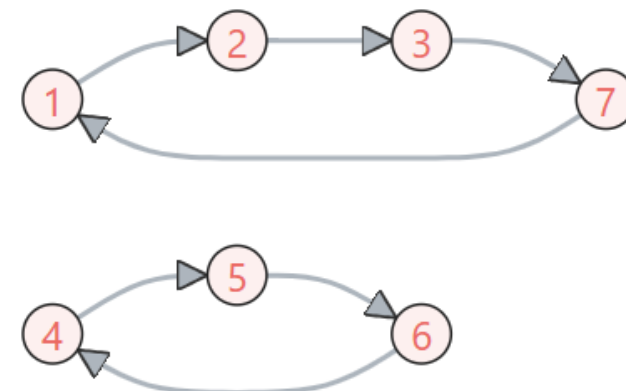
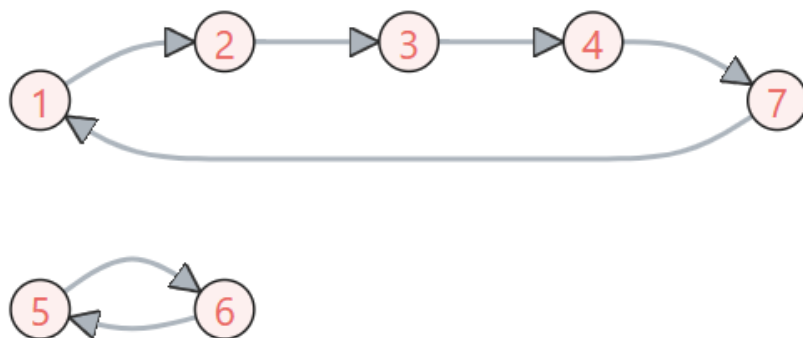
- Với mỗi đỉnh (vertex), có đúng 1 cung/cung ra (arc going out)

$$\sum_{j \in P \setminus \{i\}} x_{i,j} = 1 \quad \forall i \in P$$

- Và 1 cung vào (arc going in)

$$\sum_{j \in P \setminus \{i\}} x_{j,i} = 1 \quad \forall i \in P$$

- Hai ràng buộc (degree constraint) trên vẫn chưa đủ, vì sao?



TSP: constructing a model

■ Hai ràng buộc trên vẫn chưa đủ, vì sao?

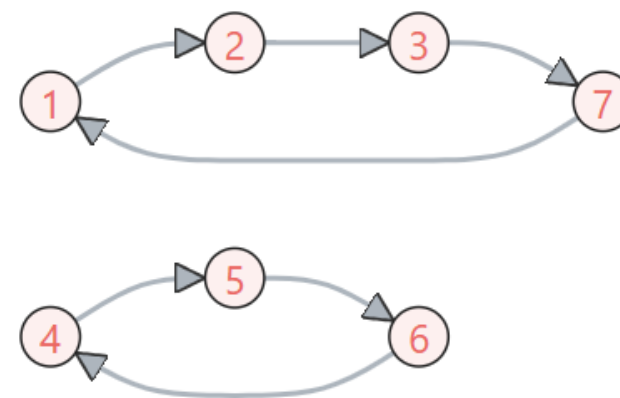
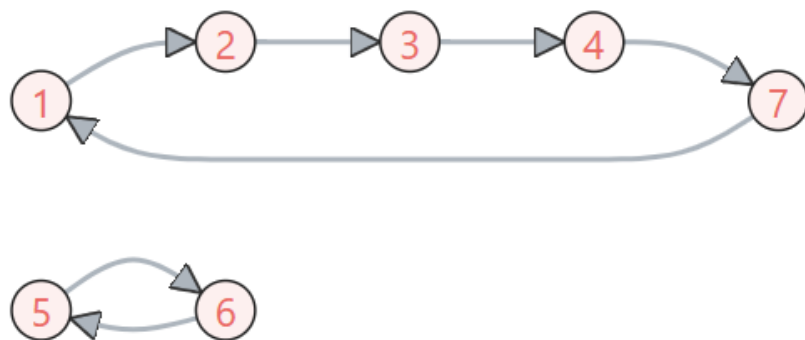
- Xét 2 chu trình con (subtour) $\{1, 2, 3, 4, 7\}$ và $\{5, 6\}$ trong đồ thị 7 đỉnh vẫn thỏa 2 ràng buộc trên (mỗi đỉnh có duy nhất 1 in và 1 out), nhưng không thỏa yêu cầu bài toán.

→ tất cả các đỉnh phải nằm trên cùng một chu trình (tour) → khử (eliminate) subtours

- Các subtour được hình thành từ 2 đến $(n - 2)$ đỉnh.

■ Ví dụ

- Subtour với 2 nodes hoặc 3 nodes



TSP: executable model

- Subtour elimination constraints (các ràng buộc loại trừ subtours)
- Quan sát
 - Một subtour (có vòng lặp/loop) bất kỳ $\rightarrow$ tổng số cạnh/cung bằng đúng số đỉnh $\rightarrow$ loại trừ/khử (eliminate) việc hình thành một subtour bằng cách ràng buộc tổng số cạnh nhỏ hơn (less than) số đỉnh của subtour.
 - Ví dụ TSP với các đỉnh $\{1, 2, 3, 4\}$, ràng buộc loại trừ subtour cho tập con (subset) $\{1, 2\}$ sẽ là (tương tự cho subtour nhiều đỉnh):
$$x_{1,2} + x_{2,1} < 2$$
 - Trong linear programming (chuyển sang $\leq$) sẽ là:
$$x_{1,2} + x_{2,1} \leq 1$$

■ Tổng quát

$$\sum_{i,j \in S} x_{i,j} \leq |S| - 1$$

TSP: executable model

■ Nhờ các degree constraint

- Một subtour ban đầu bị loại trừ đi một cạnh → để thỏa các degree constraints → phải connect với các đỉnh của subtour khác → ... → tất cả các đỉnh sẽ cùng một tour (chính là subtour lớn nhất)

■ Trick để cài đặt model

- Thực thi model không có Subtour elimination constraints (SEC)
- Nếu solver trả về
 - chỉ có một tour → dừng (đó là solution)
 - Một tập hợp S các subtours → chạy lại model với ràng buộc SEC cho từng subtour trong tập S cho đến khi chỉ trả về một subtour (khi này chính là tour cần tìm/solution).

TSP: executable model

```
7  # subtour elimination constraint
8  # D: distances matrix; subtours[[]]: 2D-list, tập các subtours
9  def solve_tsp_eliminate(D, subtours=None):
10     if subtours is None:
11         subtours = []
12     solver = pywraplp.Solver("TSP", pywraplp.Solver.CBC_MIXED_INTEGER_PROGRAMMING)
13     n = len(D)
14
15     # Decision variables
16     x = [[solver.IntVar(0, 0 if D[i][j] == 0 else 1, "x[%d,%d]" % (i, j))
17           for j in range(n)] for i in range(n)]
18
19     # Constraints
20     for i in range(n):
21         solver.Add(solver.Sum(x[i][j] for j in range(n)) == 1)
22         solver.Add(solver.Sum(x[j][i] for j in range(n)) == 1)
23         solver.Add(x[i][i] == 0)
```

TSP: executable model

```

25 # Subtour elimination constraints
26 # Một subtour bất kỳ -> tổng số cạnh/cung bằng đúng số đỉnh.
27 # Loại trừ việc hình thành subtour bằng cách ràng buộc tổng số cạnh
28 # nhỏ hơn (less than) số đỉnh của subtour.
29 for sub in subtours:
30     solver.Add(solver.Sum(x[i][j] for i in sub for j in sub) <= len(sub) - 1)
31
32 # Objective function
33 solver.Minimize(solver.Sum(x[i][j] * (0 if D[i][j] is None else D[i][j])
34                 for i in range(n) for j in range(n)))
35
36 status = solver.Solve()
37 # chuyển ma trận biến quyết định x sang ma trận giá trị X
38 X = SolVal(x)
39 # trích xuất tập các chu trình con (set of subtours) từ ma trận giá trị X
40 tours = extract_tours(X, n)
41
42 return status, ObjVal(solver), tours

```

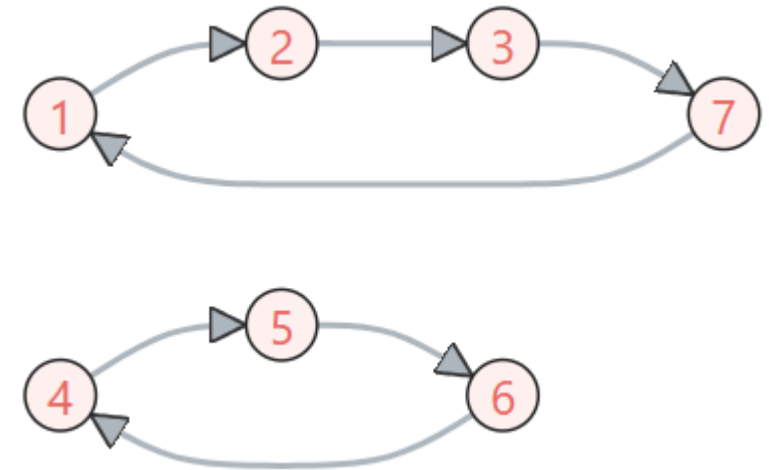
TSP: executable model

```
45 # X: ma trận giá trị các biến quyết định
46 def extract_tours(X, n):
47     node = 0
48     tours = [[0]]
49     all_nodes = [0] + [1] * (n - 1)
50
51     # lặp lại cho đến khi số lượng đỉnh/nodes còn lại = 0
52     # (nghĩa là các tours do solver trả về là 1)
53     while sum(all_nodes) > 0:
54         next_node = [i for i in range(n) if X[node][i] == 1][0]
55         if next_node not in tours[-1]:
56             tours[-1].append(next_node)
57             node = next_node
58         else:
59             node = all_nodes.index(1)
60             tours.append([node])
61             all_nodes[node] = 0
62     return tours
```

TSP: executable model

- Hàm `extract_tours()` sau khi xử lý ma trận các biến quyết định X sẽ cho ra kết quả là tập các subtours
 - Ma trận kề (adjacency matrix) X mô tả hình bên (*index bắt đầu từ 0*)
 - Subtours được sử dụng cho lần lặp kế tiếp...

```
100 # Kiểm tra hàm extract_tours()
101 X = [[0, 1, 0, 0, 0, 0, 0],
102      [0, 0, 1, 0, 0, 0, 0],
103      [0, 0, 0, 0, 0, 0, 1],
104      [0, 0, 0, 0, 1, 0, 0],
105      [0, 0, 0, 0, 0, 1, 0],
106      [0, 0, 0, 1, 0, 0, 0],
107      [1, 0, 0, 0, 0, 0, 0]]
108
109 print("Set of subtours:", extract_tours(X, 7))
```



Set of subtours: `[[0, 1, 2, 6], [3, 4, 5]]`

TSP: executable model

```
65 def solve_tsp(D):
66     subtours = []
67     tours = []
68     status, obj_val = 0, 0
69     # khi tours chỉ còn 1 tour thì dừng (đó là subtour lớn nhất)
70     while len(tours) != 1:
71         status, obj_val, tours = solve_tsp_eliminate(D, subtours)
72         if status == 0:
73             subtours.extend(tours) # add all tours to subtours
74             print("Set of subtours:", tours)
75     return status, obj_val, tours[0]
```

TSP: executable model

```
78 def main():
79     # Nguồn dữ liệu https://developers.google.com/optimization/routing/tsp
80     # tsp_1.csv: undirected graph, 13 cities/nodes, 78 edges/arcs
81     # Đọc ma trận khoảng cách từ file csv
82     D = read_int_matrix("input/tsp_1.csv")
83     print('Processing...')
84     # dict tên các thành phố
85     cities = {0: 'New York', 1: 'Los Angeles', 2: 'Chicago', 3: 'Minneapolis', 4: 'Denver',
86              | 5: 'Dallas', 6: 'Seattle', 7: 'Boston', 8: 'San Francisco', 9: 'St. Louis',
87              | 10: 'Houston', 11: 'Phoenix', 12: 'Salt Lake City'}
88
89     status, obj_val, tour = solve_tsp(D)
90     print("status:", status)
91     print("Total distances:", obj_val, "miles")
92     print("Route:", tour)
93
94     # In ra tên các thành phố theo tour
95     print("Route:")
96     for i in range(len(tour)):
97         print(cities[tour[i]], end=' -> ')
98     print(cities[tour[0]])
```

TSP: executable model



```
data["distance_matrix"] = [  
    [0, 2451, 713, 1018, 1631, 1374, 2408, 213, 2571, 875, 1420, 2145, 1972],  
    [2451, 0, 1745, 1524, 831, 1240, 959, 2596, 403, 1589, 1374, 357, 579],  
    [713, 1745, 0, 355, 920, 803, 1737, 851, 1858, 262, 940, 1453, 1260],  
    [1018, 1524, 355, 0, 700, 862, 1395, 1123, 1584, 466, 1056, 1280, 987],  
    [1631, 831, 920, 700, 0, 663, 1021, 1769, 949, 796, 879, 586, 371],  
    [1374, 1240, 803, 862, 663, 0, 1681, 1551, 1765, 547, 225, 887, 999],  
    [2408, 959, 1737, 1395, 1021, 1681, 0, 2493, 678, 1724, 1891, 1114, 701],  
    [213, 2596, 851, 1123, 1769, 1551, 2493, 0, 2699, 1038, 1605, 2300, 2099],  
    [2571, 403, 1858, 1584, 949, 1765, 678, 2699, 0, 1744, 1645, 653, 600],  
    [875, 1589, 262, 466, 796, 547, 1724, 1038, 1744, 0, 679, 1272, 1162],  
    [1420, 1374, 940, 1056, 879, 225, 1891, 1605, 1645, 679, 0, 1017, 1200],  
    [2145, 357, 1453, 1280, 586, 887, 1114, 2300, 653, 1272, 1017, 0, 504],  
    [1972, 579, 1260, 987, 371, 999, 701, 2099, 600, 1162, 1200, 504, 0],  
]
```

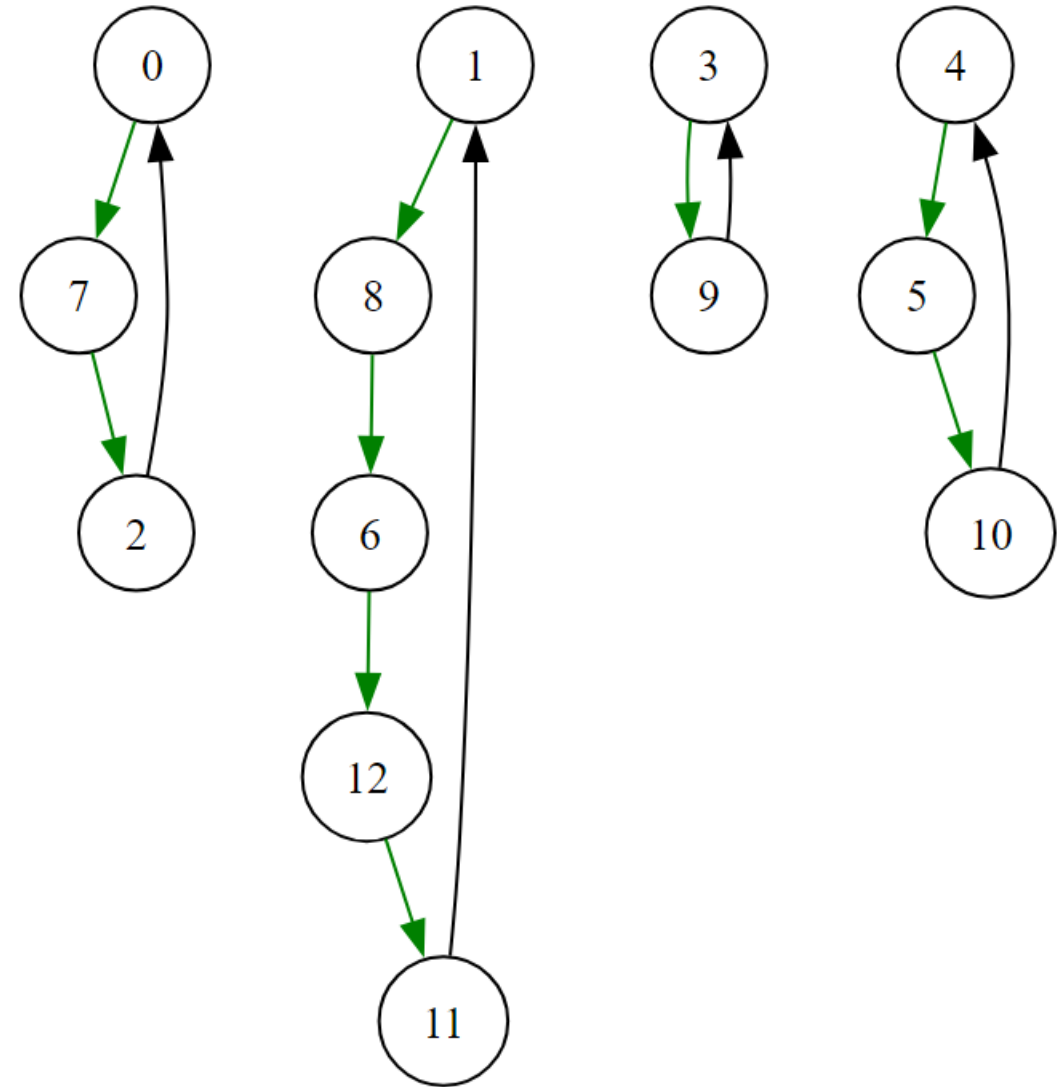

TSP: executable model



```
D:\Codes\Python\Optimization>python -u "d:\Codes\Python\Optimization\tsp_1.py"
Processing...
Set of subtours: [[0, 7], [1, 11], [2, 9, 3], [4, 12], [5, 10], [6, 8]]
Set of subtours: [[0, 2, 3, 7], [1, 8, 6, 12, 4, 11], [5, 10, 9]]
Set of subtours: [[0, 7, 2], [1, 8, 6, 12, 11], [3, 9], [4, 5, 10]]
Set of subtours: [[0, 7, 2, 3, 9], [1, 11, 10, 5, 4, 12, 6, 8]]
Set of subtours: [[0, 9, 5, 10, 11, 1, 8, 6, 12, 4, 3, 2, 7]]
status: 0
Total distances: 7293.0 miles
Route: [0, 9, 5, 10, 11, 1, 8, 6, 12, 4, 3, 2, 7]
Route:
New York -> St. Louis -> Dallas -> Houston -> Phoenix -> Los Angeles -> San Francisco
-> Seattle -> Salt Lake City -> Denver -> Minneapolis -> Chicago -> Boston -> New York
```

TSP: executable model

- Một trong các tập hợp các subtours
 - Set of subtours:
 $[[0, 7, 2], [1, 8, 6, 12, 11], [3, 9], [4, 5, 10]]$



TSP: executable model

■ Directed graph

Table 5-9. *Example of Distance Matrix for TSP*

P (x y)	P0	P1	P2	P3	P4	P5	P6	P7	P8	P9
P0 72 19		711	107	516	387	408	539	309	566	771
P1 10 37	539		769	881	380	546	655	443	295	1140
P2 77 31	122	752		281	441	264	318	448	588	730
P3 89 61	519	875	274		435	334	93	776	949	302
P4 51 61	484	561	338	419		118	268	607	495	431
P5 57 52	409	406	244	380	93		295	544	549	494
P6 82 69	479	735	334	101	345	247		679	809	238
P7 52 1	221	444	433	744	487	435	649		325	840
P8 21 14	510	303	599	984	531	553	847	350		1001
P9 88 96	663	989	664	335	588	434	297	1093	1012	

TSP: executable model

```
D:\Codes\Python\Optimization>python -u "d:\Codes\Python\Optimization\tsp.py"
Processing...
Set of subtours: [[0, 2], [1, 7, 8], [3, 6, 9], [4, 5]]
Set of subtours: [[0, 2, 7], [1, 8], [3, 6], [4, 9, 5]]
Set of subtours: [[0, 2, 3, 6, 9, 5, 4, 1, 8, 7]]
status: 0
obj_val: 2673.0
tour: [0, 2, 3, 6, 9, 5, 4, 1, 8, 7]
```

Table 5-10. Successive Iterations of the TSP Solver Showing Optimal Values and Subtours

Iter (value)	Tour(s)
0-(2177)	[0, 2]; [1, 7, 8]; [3, 6, 9]; [4, 5]
1-(2526)	[0, 2, 7]; [1, 8]; [3, 6]; [4, 9, 5]
2-(2673)	[0, 2, 3, 6, 9, 5, 4, 1, 8, 7]

TSP: variations

- TSP là một trong các bài toán tổ hợp được nghiên cứu nhiều nhất (most studied combinatorial problems)
- Biến thể đơn giản
 - Tìm đường đi qua tất cả các đỉnh (simple path) thay vì tour (closed path)
 - Bài toán TSP-P (TSP-Path)
 - *[Đọc thêm textbook]*

